

```

import torch

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print(
        "GPU Memory:",
        round(torch.cuda.get_device_properties(0).total_memory /
1024**3, 2),
        "GB"
    )
else:
    print("WARNING: GPU is not available.")

PyTorch version: 2.11.0+cu128
CUDA available: True
GPU: Tesla T4
GPU Memory: 14.56 GB

!pip install -q -U diffusers transformers accelerate safetensors
gradio

_____ 5.9/5.9 MB 81.4 MB/s eta
0:00:00
_____ 11.7/11.7 MB 115.1 MB/s eta
0:00:00
_____ 31.3/31.3 MB 71.2 MB/s eta
0:00:00
_____ 62.6/62.6 kB 6.4 MB/s eta
0:00:00

import os
import gc
import random
from datetime import datetime

import torch
import gradio as gr

from diffusers import AutoPipelineForText2Image

[transformers] `Siglip2ImageProcessorFast` is deprecated. The `Fast`
suffix for image processors has been removed; use
`Siglip2ImageProcessor` instead.

MODEL_ID = "stable-diffusion-v1-5/stable-diffusion-v1-5"

print("Loading Stable Diffusion v1.5...")
print("This may take a few minutes the first time.")

```

```
pipe = AutoPipelineForText2Image.from_pretrained(
    MODEL_ID,
    torch_dtype=torch.float16,
    variant="fp16",
    use_safetensors=True
)
```

```
pipe = pipe.to("cuda")
```

```
print("🟢 Model loaded successfully!")
```

Loading Stable Diffusion v1.5...

This may take a few minutes the first time.

/usr/local/lib/python3.13/dist-packages/huggingface_hub/utils/_validators.py:205: UserWarning: The `local\_dir\_use\_symlinks` argument is deprecated and ignored in `hf\_hub\_download`. Downloading to a local directory does not use symlinks anymore.

```
warnings.warn(
```

```
{"model_id": "009fd0c0233e46d0bf6833fb8a9d7ccf", "version_major": 2, "version_minor": 0}
```

/usr/local/lib/python3.13/dist-packages/diffusers/utils/deprecation_utils.py:23: FutureWarning: `torch\_dtype` is deprecated and will be removed in version 1.0.0. Please use `dtype` instead.

```
deprecate("torch_dtype", "1.0.0", _TORCH_DTYPE_DEPRECATION_MESSAGE)
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```
{"model_id": "35583b85f59245738df25339b5e516ba", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "dbfd7abd11624ddc815beef25b8357f9", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "7003418a624645a286b4c9e3930b3cf8", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "80cd0b99d8b2438ab899e11b351fa509", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "63240dd3f44f4e94811888fcc3a0061e", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "bca8c5e9f3634b0087d144b21f8a7c26", "version_major": 2, "version_minor": 0}
```

□ Model loaded successfully!

```
prompt = "A beautiful futuristic city at sunset, flying cars,  
cinematic lighting, highly detailed"
```

```
negative_prompt = "blurry, low quality, distorted, deformed"
```

```
generator = torch.Generator(device="cuda").manual_seed(42)
```

```
image = pipe(  
    prompt=prompt,  
    negative_prompt=negative_prompt,  
    num_inference_steps=30,  
    guidance_scale=7.5,  
    generator=generator  
) .images[0]
```

```
display(image)
```

```
{"model_id": "88f9b62552174b6f8df5e9ad1b6e5f93", "version_major": 2, "version_minor": 0}
```



```
os.makedirs("generated_images", exist_ok=True)
print("[] generated_images folder ready")
[] generated_images folder ready
def generate_image(
    prompt,
    negative_prompt,
    steps,
    guidance_scale,
    seed
):
```

```

# Check prompt
if prompt is None or not prompt.strip():
    raise gr.Error("Please enter a prompt.")

try:
    # Convert values
    steps = int(steps)
    guidance_scale = float(guidance_scale)
    seed = int(seed)

    # Create reproducible generator
    generator = torch.Generator(
        device="cuda"
    ).manual_seed(seed)

    # Generate image
    with torch.inference_mode():

        result = pipe(
            prompt=prompt,
            negative_prompt=negative_prompt,
            num_inference_steps=steps,
            guidance_scale=guidance_scale,
            generator=generator
        )

        image = result.images[0]

    # Create timestamp
    timestamp = datetime.now().strftime(
        "%Y%m%d_%H%M%S"
    )

    # Save image
    filename = (
        f"generated_images/"
        f"image_{timestamp}_{seed}.png"
    )

    image.save(filename)

    print(f"🖼 Image saved: {filename}")

    return image

except Exception as e:
    print("Generation error:", str(e))

    raise gr.Error(

```

```

        f"Image generation failed: {str(e)}"
    )
with gr.Blocks(
    title="Generative AI Image Generator"
) as demo:
    gr.Markdown(
        """
        # 🎨 Generative AI Image Generator

        ### Generate images from natural-language prompts

        This application uses a pre-trained
        **Stable Diffusion v1.5** model to generate
        images from user-provided text prompts.
        """
    )

    with gr.Row():

        # =====
        # LEFT SIDE - INPUTS
        # =====

        with gr.Column():

            prompt = gr.Textbox(
                label="📝 Image Prompt",
                placeholder=(
                    "Describe the image you want to generate..."
                ),
                lines=5
            )

            negative_prompt = gr.Textbox(
                label="🚫 Negative Prompt",
                value=(
                    "blurry, low quality, distorted, "
                    "deformed, ugly"
                ),
                lines=3
            )

            steps = gr.Slider(
                minimum=10,
                maximum=50,
                value=30,
                step=1,
                label="🔧 Inference Steps"
            )

```

```

    )

    guidance_scale = gr.Slider(
        minimum=1,
        maximum=15,
        value=7.5,
        step=0.5,
        label="📏 Guidance Scale"
    )

    seed = gr.Number(
        value=42,
        precision=0,
        label="🌱 Seed"
    )

    generate_button = gr.Button(
        "🖼️ Generate Image",
        variant="primary"
    )

    # =====
    # RIGHT SIDE - OUTPUT
    # =====

    with gr.Column():

        output_image = gr.Image(
            label="Generated Image",
            type="pil"
        )

    # =====
    # BUTTON EVENT
    # =====

    generate_button.click(
        fn=generate_image,
        inputs=[
            prompt,
            negative_prompt,
            steps,
            guidance_scale,
            seed
        ],
        outputs=output_image
    )

demo.launch(
    share=True,

```

```
    debug=True  
)
```

Colab notebook detected. This cell will run indefinitely so that you can see errors and logs. To turn off, set debug=False in launch().

* Running on public URL: <https://4clffcf2383f94acae.gradio.live>

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to deploy to Hugging Face Spaces (<https://huggingface.co/spaces>)

<IPython.core.display.HTML object>

```
{"model_id": "3eceba05a65a476f9277f4a1f1af39e6", "version_major": 2, "version_minor": 0}
```

Image saved: generated_images/image_20260825_093343_42.png

```
{"model_id": "20f263995845486e933f1b98f7e9da28", "version_major": 2, "version_minor": 0}
```

Image saved: generated_images/image_20260825_093406_42.png

```
{"model_id": "91d1ee2a6ea94fef35657de063c0a36", "version_major": 2, "version_minor": 0}
```

Image saved: generated_images/image_20260825_094247_42.png